TillPOS

Developer Guide

Setup, dev environment, code map, local database, PINs and sync

Designed and Developed By **M.Babar**
+92 315 6500221

Version 1.0.1 · 13 October 2026

TillPOS is the offline-first till (cash counter) for QuickGroc. It is a Windows desktop app (.NET 10, WPF) that:

- downloads items, prices, taxes, cashiers and POS settings from **ERPNext** (POS Awesome) into a local SQLite database,
- sells, returns, holds and delivers **offline** from that local database,
- uploads shifts, bills (POS Invoices) and supervisor approvals to ERPNext in the background.

This guide takes a new developer from an empty PC to a running dev till, and explains where each process lives in the code.

> **Safety rules** - **Production** (`https://erp.quickgroc.com`) is the live shop. Never test against it, and never write to it without the owner's approval. - **Dev** (`https://dev.quickgroc.com`) is a copy of production where writes are allowed. All testing happens there. - Never commit keys, secrets or PINs. Everything under `publish/` and every `*.db` file is git-ignored on purpose.

---

1. Set up your PC

Tool	Version	Used for
Windows 10/11	x64	The app is WPF (Windows only).
.NET SDK	**10.x**	Building and running everything. Check with `dotnet --version`.
Git	any recent	Getting the code.
Node.js	18+	Only for `tools/erpnext-setup.mjs` (ERPNext setup script).
Visual Studio 2022 (17.14+) / VS Code with C# Dev Kit / Rider	–	Editing and debugging.
DB Browser for SQLite (optional)	–	Looking inside the till's local database.

2. Get the code

```
git clone https://github.com/MBS-AI-PMO/ERP-NEXT-POS-APP.git ERP-NEXT
cd ERP-NEXT
git checkout master
```

Repository layout:

```
ERP-NEXT/
├─ tillpos/                ← the till app (this guide)
│  ├─ TillPOS.slnx         ← solution (open this in the IDE)
│  ├─ Directory.Build.props ← net10.0, nullable, warnings are errors
│  ├─ src/                 ← the projects (see section 6)
│  ├─ tests/TillPOS.Tests/  ← xUnit tests
│  └─ tools/               ← setup script, CLI, build scripts, START HERE templates
├─ docs/                   ← design spec, plans, ERPNext setup checklist, API notes
├─ deploy/erp-local/       ← how to run a local copy of ERPNext (WSL2 + bench)
└─ publish/                ← build outputs and credential files (git-ignored, never committed)
```

Read these next: `docs/superpowers/specs/2026-10-05-tillpos-offline-pos-design.md` (master design), `docs/erpnext-production-setup.md` (what ERPNext needs), and the plans in `docs/superpowers/plans/` (history of every feature).

3. Build and run the tests

```
cd tillpos
dotnet build
dotnet test -m:1
```

- `-m:1` builds one project at a time. Without it, parallel builds sometimes lock files on Windows.
- About 1,000 tests run in roughly 10–15 seconds. They need **no** ERPNext and no internet: ERPNext, the database, the clock, dialogs and the printer are all faked.
- `TreatWarningsAsErrors` is on: a compiler warning fails the build. Fix it, don't suppress it.

All tests must pass before every commit.

4. Set up the dev environment (ERPNext)

4.1 Get a dev API user

Ask the owner for **your own** API user on dev (`https://dev.quickgroc.com`) with: - roles **TillPOS Device**, **POS User**, **Accounts User**, **Sales User**, **Stock User**; - an API key and secret (ERPNext → User → *API Access* → *Generate Keys*).

Do **not** use the shared `till-dev@quickgroc.com` user. ERPNext allows only **one open shift per user per POS Profile**. If two people's tills use the same user, one person's open shift blocks the other's.

4.2 What dev already has

Dev is a copy of production. These were created by `tools/erpnext-setup.mjs`:

- **POS Profiles** (counters): *Al Ain Counter 1*, *Al Ain Counter 2*, *Test Counter*.
- Custom fields: `POS Invoice-custom_till`, `POS Opening Shift-custom_offline_id`, `POS Closing Shift-custom_offline_id`.
- DocType **POS Cashier**: cashier name, linked user, **PIN**, supervisor flag, enabled.
- DocType **TillPOS Approval**: the log of supervisor actions sent by the tills.

- Mode of Payment **Rounding**: used when a card payment is rounded up.
- Test cashiers **Sandbox Cashier** and **Sandbox Supervisor**. Ask the owner for their PINs. Never use real cashiers' PINs.

To set up another ERPNext site (for example a local one from `deploy/erp-local/`), run the setup script with a System Manager key. It is safe to run again, because it creates only what is missing:

```
node tillpos/tools/erpnext-setup.mjs path\to\config.json
```

Config fields: `BaseUrl`, `ApiKey`, `ApiSecret`, optional `TillRole` (default "TillPOS Device") and optional `Cashiers[]` (`cashier_name`, `pin`, `is_supervisor`, `user`). Keep the file under `publish/` so it is never committed.

5. Run the till from source

5.1 A dev settings file

The till reads its settings from `C:\ProgramData\TillPOS\settings.json`, unless the environment variable `TILLPOS_SETTINGS` points at another file. For development, always use your own file and your own database, so you never touch a real till's data.

Create `publish\dev-<yourname>\settings.json`:

```
{
  "Environment": "Dev",
  "BaseUrl": "https://dev.quickgroc.com/",
  "ApiKey": "<your dev api key>",
  "ApiSecret": "<your dev api secret>",
  "TillNumber": 9,
  "DbPath": "",
  "Counters": [
    { "PosProfile": "Test Counter", "Label": "Test Counter", "CashMode": "Cash", "CardMode": "Card"
}
  ],
  "Upload": "DryRun",
  "SetupDone": false
}
```

- `"DbPath": ""` puts `till.db` next to this settings file.
- On first start the till encrypts the secret (Windows DPAPI). The file then holds `ApiSecretProtected` and the plain secret is removed.
- Use a **TillNumber** no real till uses (real tills are 1 and 2). Bill ids start with `TILL<n>-`.
- Use the counter names exactly as they appear in ERPNext, and the Mode of Payment names that POS Profile uses.

5.2 Start it

```
$env:TILLPOS_SETTINGS = "D:\...\ERP-NEXT\publish\dev-<yourname>\settings.json"
dotnet run --project tillpos/src/TillPOS.App
```

In Visual Studio, set the same variable in the TillPOS.App launch profile (*Debug → Environment variables*).

What happens on first start: 1. **Setup screen** (`SetupDone` false): choose a printer (leave it empty to write receipts as text files into `receipts\` next to the database), the till number, the counters and the upload mode. 2. **Download screen**: the first full catalog download. It retries every 30 s until the POS settings arrive. 3. **Login**: log in with a Sandbox cashier PIN, open a shift (choose the counter, enter the float), sell.

Only one TillPOS can run at a time on a PC (single-instance lock). Close any installed till first.

5.3 Upload modes

Mode	What it does	When to use
`Off`	Nothing is sent. Bills wait in the local outbox.	Default for a new production till.
`DryRun`	Builds every document, checks it against ERPNext **read-only**, writes the JSON to `outbox-preview\` next to the database. Nothing is created.	Checking payloads.
`Live`	Creates and submits the shifts, POS Invoices and approvals in ERPNext.	Dev only while developing. On production, only on go-live.

Live is only allowed for a Dev build pointing at `https://dev.quickgroc.com`, or a Production build without test settings (`TillSettings.LiveAllowed`). Any other combination falls back to Off.

The **first** switch to Live records the moment in `kv.upload_live_since`. Shifts closed before it count as test data and are **excluded**. The first switch is refused while a shift is open.

6. How the code is organised

Dependencies: `Core` ← `Data`, `Erp`, `Printing` ← `Sync` ← `Presentation` ← `App`.

Project	Responsibility	Has UI?	Talks to ERPNext?
`TillPOS.Core`	Business rules: cart, pricing, tax, payments, rounding, returns, shifts, PINs. Pure C#, no I/O.	no	no
`TillPOS.Data`	The local SQLite database: schema migrations and every store.	no	no
`TillPOS.Erp`	HTTP client for the ERPNext REST API (read-only interface plus a separate writer).	no	yes
`TillPOS.Sync`	Download feeds (ERPNext → till) and the Uploader (till → ERPNext).	no	yes
`TillPOS.Printing`	Receipt, delivery slip and Z-report rendering (ESC/POS), FTA QR code.	no	no
`TillPOS.Presentation`	View models (MVVM, CommunityToolkit.Mvvm): every screen's logic. Fully unit-tested.	no	via Sync
`TillPOS.App`	WPF views, dialogs, settings, printer output, start-up (`AppHost`), the background `SyncService`.	yes	via Sync
`tools/TillPOS.SyncCli`	Console tool: pull the catalog, scan barcodes, compare totals with ERPNext.	no	yes

6.1 Start-up (`src/TillPOS.App`)

File	What it does
`App.xaml.cs`	Entry point: single-instance lock, loads settings (`SettingsStore`), shows the Setup dialog, starts `AppHost`, writes `errors.log`.
`TillSettings.cs`	The settings record (all fields and defaults; `LiveAllowed`, `EffectiveUpload`, `EffectiveCounters`).
`SettingsStore.cs`	Where settings come from: `TILLPOS_SETTINGS` → `C:\ProgramData\TillPOS\settings.json` → `settings.json` beside the exe → settings built into the exe. Encrypts the secret.
`AppHost.cs`	Builds everything: database, stores, ERPNext client, uploader, `TillContext`, and starts `SyncService`.
`SyncService.cs`	Background loop: full sync every 90 s, price check every 15 s, upload soon after each sale or shift change.
`MainWindow.xaml`	Shell window; maps each view model to its view (DataTemplates).
`WpfDialogs.cs`	Implements `IDialogs` (PIN pad, number pad, message boxes, setup, sync status).
`ReceiptOutput.cs`, `RawPrinter.cs`	Sends receipts to the Windows printer (raw ESC/POS), or writes text files when no printer is set.

6.2 Screens: which file handles which process

View models are in `src/TillPOS.Presentation`, views in `src/TillPOS.App/Views` (dialogs in `src/TillPOS.App/Dialogs`).

Process	View model	View	Main logic it uses
First download	`DownloadViewModel`	`DownloadView`	`CatalogPuller`
Login with PIN	`LoginViewModel`	`LoginView`	`Authenticator`, `PinAttemptLimiter`
Open shift (counter, float)	`OpenShiftViewModel`	`OpenShiftView`	`ShiftStore`, `CounterSettings`
Sale: scan, search (F2), qty (F3), price check (F4), hold/recall (F5/F7), void (F8), delivery (F9), update prices (F10)	`SaleViewModel` (+ `PriceCheckViewModel`, `HeldBillsViewModel`)	`SaleView` (keys F2/F10/Del/Esc in `SaleView.xaml.cs`)	Core `Cart`, `SaleContext`, `ScanBuffer`, `ScaleLabel` (weighed-item barcodes)
Payment: cash F12, card F11, split	`PaymentViewModel`	`PaymentView`	`PaymentCalculator`, `SaleRecorder`, `ReceiptOutput`
Returns (F6), with or without receipt, also other tills' bills	`ReturnViewModel`	`ReturnView`	`ReturnBuilder`, `RemoteReceiptStore`, `RemoteReturnsCheck`
Deliveries: unpaid delivery bills, paid later	`DeliveriesViewModel`	`DeliveriesView`	`DeliveryStore`, Core `Delivery`

Process	View model	View	Main logic it uses
Close shift: blind count, supervisor PIN, Z report	`CloseShiftViewModel`	`CloseShiftView`	`ShiftCalculator`, `ShiftReportRenderer`
Supervisor PIN for protected actions	`SupervisorGate`	`Dialogs/PinDialog`	`Authenticator.Supervisor`, `ApprovalStore`
Sync status (header pill)	`SyncStatusViewModel`	`Dialogs/SyncStatusDialog`	`Uploader` counts, last `PullReport`
Upload problems: retry, mark handled, include	`UploadProblemsViewModel`	`UploadProblemsView`	`Uploader`, the stores' upload state
Header: shop, till, cashier, online, waiting/failed	`ShellViewModel`	`MainWindow`	–

6.3 Business rules (`src/TillPOS.Core`)

Folder	Main classes	Purpose
`Catalog`	`ICatalog`, `PosSettings`, item models	What the till knows about items and the POS profile.
`Pricing`	`PriceResolver`, `PricingRuleSelector`, `LineMath`	Price list rate, pricing rules, discounts.
`Tax`	`TaxCalculator`, `ItemTaxResolver`	VAT, ported from ERPNext's `taxes_and_totals` so totals match ERPNext.
`Money`	`Rounder`, `RoundingMethod`	Rounding exactly like Frappe (`flt` / `rounded`).
`Payments`	`PaymentCalculator`	Cash is rounded to 0.25; card is charged exactly; split payments.
`Sales`	`Cart`, `Receipt`, `ClientIds`, `SaleRecorder`, `ReturnBuilder`, `ScaleLabel`, `HeldCart`, `Delivery`	The bill and everything saved from it.
`Shifts`	`ShiftOpening`, `ShiftClosing`, `ShiftCalculator`, `CounterSettings`	Shifts and the blind cash count.
`Security`	`PinHasher`, `Authenticator`, `PinAttemptLimiter`, `ApprovalRecord`, `ApprovalAction`	PINs and the supervisor audit trail.

7. The local database

7.0 Where the database comes from (it is not in Git)

`*.db` files are git-ignored on purpose. A database holds one till's own bills, shifts and hashed PINs, and the catalog changes every day, so a copy in Git would be wrong for everyone else. **No database file is needed.** The till builds its own:

1. **Schema:** on start `AppHost` opens the path in `DbPath` (or `till.db` next to the settings file). If the file doesn't exist, SQLite creates an empty one, and `TillDb.Migrate()` runs every migration in `src/TillPOS.Data/Migrations.cs`, so all tables appear.
2. **Data:** the **Download** screen then pulls everything from ERPNext (POS Profiles, items, barcodes, prices, taxes, pricing rules, cashiers). For dev that is `https://dev.quickgroc.com`, which takes a few minutes the

first time.
3. **Ready:** log in with a Sandbox cashier PIN. Bills, shifts and approvals you create are added to that same file.

So a developer who has just pulled from Git only needs the dev **settings file** with their own API key (section 5.1). The database follows on the first run.

- **Start again from empty (dev only):** close the till, delete your dev `till.db` (plus `till.db-wal` and `till.db-shm`), and start again. Only delete it while nothing is waiting to upload, or those bills are lost.
- **Tests** never use your database: each test makes its own temporary migrated file (`Data/TempDb.cs`).
- **Changing the schema:** add a migration (see below). Every existing till upgrades itself on its next start.
- **Looking at a real till's data:** ask for a copy of its `till.db` together with `till.db-wal`, and open the copy read-only. Never run a dev till on a copy of a production database: it would try to upload that till's bills.

One SQLite file per till: `C:\ProgramData\TillPOS\till.db` on a real till (your dev file sits next to your dev settings). Errors go to `errors.log` and receipts without a printer go to `receipts\`, both in the same folder.

- **Opened by** `TillDb` (`src/TillPOS.Data/TillDb.cs`) in WAL mode with `synchronous=FULL`, so a power cut never loses a saved bill.
- **Schema versions:** `src/TillPOS.Data/Migrations.cs` holds a list of migrations. `PRAGMA user_version` tells which ones ran. On start `Migrate()` runs the missing ones in a transaction. **To change the schema, add a new migration at the end. Never edit a migration that has shipped.** A till refuses a database newer than its own build.

7.1 Tables

Table	Holds	Written by
`item`, `item_fts`, `item_barcode`, `item_uom`, `item_tax`	Items, search index, barcodes, units, item tax	`ItemFeed` → `CatalogStore`
`item_price`	Prices of the counter's price list	`ItemPriceFeed`
`item_group`, `pricing_rule`, `item_tax_template`, `sales_tax_template`	Groups and rules (JSON)	the matching feeds
`cashier`	Cashiers with **hashed** PINs	`CashierFeed` → `CashierStore`
`shift`	Each shift: opening/closing JSON plus the upload state of the opening and of the closing	`ShiftStore`
`receipt`	**The outbox of bills** (sales and returns): JSON, `sync_status`, `erp_name`, `last_error`, attempts	`ReceiptStore`
`approval_log`	Supervisor actions and their upload state	`ApprovalStore`
`held_cart`	Held bills (F5)	`HeldCartStore`
`delivery`	Delivery bills not yet paid	`DeliveryStore`
`remote_receipt`	Other tills' recent bills (for cross-till returns), read-only	`RecentInvoicesFeed` → `RemoteReceiptStore`
`kv`	Key/value state, see below	various

Useful `kv` keys: - `pos_settings` and `pos_settings:<profile>`: the downloaded POS Profile settings. - `sync_mark:<feed>`: where each download feed stopped (incremental sync). - `upload_live_since`: when the till first went Live. - `current_cart`: the open bill, autosaved so a crash loses nothing. - `last_receipt`, `last_counter`.

Upload states (`sync_status` and the shift's `opening_status`/`closing_status`): `Pending` → `Synced`, or `Failed` (retried with a 30 s → 5 min backoff), `Excluded` (test data from before going Live), `Handled` (a supervisor fixed it by hand in ERPNext).

7.2 Looking inside

Close the till first, or open a **copy** of `till.db` together with its `-wal` file. Read-only queries:

```
-- what is waiting / failed
SELECT sync_status, COUNT(*) FROM receipt GROUP BY sync_status;
SELECT client_id, opened_at, closed_at, opening_status, erp_opening, closing_status, erp_closing,
opening_error
FROM shift ORDER BY opened_at;
SELECT client_id, last_error FROM receipt WHERE sync_status = 'Failed';
-- going-live moment
SELECT * FROM kv WHERE key = 'upload_live_since';
```

Never edit a real till's database by hand. Fix data through **Upload problems** on the till, or in ERPNext.

8. Cashiers and PINs

- Cashiers are maintained **in ERPNext** in the **POS Cashier** list: name, linked user, PIN (4–6 digits), *Is Supervisor*, *Enabled*.
- The PIN field is at permission level 1, so only roles allowed level 1 (System Manager, the till role) can see it.
- The till downloads the list with each full sync (about every 90 s). New cashiers and PIN changes work **without a restart**.
- On the till, PINs are stored only as hashes: PBKDF2-SHA256, 10,000 rounds, random salt (`PinHasher` in `src/TillPOS.Core/Security/Security.cs`). A PIN can't be read back from the till.
- Login rules (`Authenticator`, `PinAttemptLimiter`):
- only enabled cashiers can log in;
- a PIN shared by two cashiers logs **nobody** in (`CashierFeed` reports shared PINs);
- 5 wrong PINs in a row lock the pad for 60 seconds.
- **Supervisor PIN** (`SupervisorGate`) is needed for:
- removing a line or lowering a quantity, and voiding a bill;
- deleting a held bill;
- returns without a receipt, over the limit, of old receipts, or of another till's bill while offline;
- closing a shift;
- changing or cancelling deliveries;
- settings and upload mode;
- Upload problems actions.

Every approval, and every wrong supervisor PIN, is logged and uploaded as a **TillPOS Approval**. - **Checking a PIN problem:** 1. In ERPNext, open POS Cashier: is the cashier *Enabled*, is the PIN 4–6 digits, and does another cashier have the same PIN? 2. On the till, open Sync status → Download: does the *POS Cashier* feed say OK? 3.

Wait one full sync (about 90 s) and try again. - **Local test cashiers:** `LocalTestCashiers` in settings works only until ERPNext cashiers have synced. Test builds only.

9. Sync: how data moves

9.1 Download (ERPNext → till)

`CatalogPuller.CreateDefault` (`src/TillPOS.Sync/CatalogPuller.cs`) runs these feeds in order:

1. POS Profile (each counter)
2. Item Group
3. Item Tax Template
4. Sales Taxes and Charges Template
5. Pricing Rule
6. Item (with barcodes, units and taxes)
7. Item Price
8. Deleted Document
9. Reconcile (daily clean-up of items the server no longer has)
10. POS Cashier
11. other tills' recent bills

Incremental: `KeysetPager` asks only for rows changed since the last mark (`modified >= mark`, 500 per page), saving the mark after each page. Feeds must be idempotent, because each pull steps back 5 minutes.

Timing (`SyncService`): - full sync every `SyncIntervalSeconds` (90 s); - **prices only every 15 s**; - **F10** on the sale screen fetches prices at once.

New prices also re-price the open bill.

9.2 Upload (till → ERPNext)

`Uploader` (`src/TillPOS.Sync/Upload/Uploader.cs`), per shift, oldest first:

1. **POS Opening Shift**: waits while an earlier shift of the same counter is still open in ERPNext.
2. **POS Invoices** of that shift, oldest first. A return waits for its original sale.
3. **POS Closing Shift**: only after the shift is closed on the till **and every bill is in ERPNext**.
4. **TillPOS Approvals**.

Details: - Documents are built in `Upload/Payloads.cs`. - Each document carries the till's own id: `posa_client_request_id` for bills (`TILL2-20261009143455-000075`), and `custom_offline_id` for shifts (`TILL2-SHIFT-…`) and approvals. The uploader looks the id up first, so a document is **never created twice**, even after a lost answer. - Invoices go in as drafts. Their totals are compared with the till's, and only then submitted. - Write guard: the app uses a read-only client everywhere. A writer exists only in Live mode where Live is allowed (`UploadPipeline.LiveWriter`).

10. Building packages

Script	Builds	Reads (git-ignored)
`tools/publish-release.ps1 -Version 1.0.1 [-Till 1|2]`	Production `publish/TillPOS-<v>-till<n>/TillPOS.exe` (one self-contained exe per till) plus START HERE. Upload Off.	`publish/TillPOS.Production/till<n>.json`
`tools/publish-field.ps1 -Environment Dev [-SingleExe]`	Dev or test field packages.	`publish/TillPOS.Sandbox/till-dev.json` (Dev) or `publish/TillPOS.SyncCli/tillpos.cli.json` (Production test)

```
cd tillpos
powershell -ExecutionPolicy Bypass -File tools\publish-release.ps1 -Version 1.0.2
```

Steps for a release: 1. Bump `<Version>` in `src/TillPOS.App/TillPOS.App.csproj`. 2. Update `tools/release/START HERE.txt` if cashiers need to know something new. 3. Run the tests, then build.

Updating a till: close TillPOS, replace only `TillPOS.exe`, and start it.

Never delete or rename `C:\ProgramData\TillPOS`. It holds the bills that are still waiting, the shifts and the Live state. A till that starts with an empty folder forgets its open shifts. Its next shift opening is then refused by ERPNext ("already has an open POS shift ...") until that shift is closed in ERPNext by hand.

SyncCli (optional)

```
cd tillpos/tools/TillPOS.SyncCli
# tillpos.cli.json (git-ignored): BaseUrl, ApiKey, ApiSecret, PosProfile, DbPath, Precision,
Rounding, AllowWrites
dotnet run -- pull               # full catalog download, rows/time per feed
dotnet run -- scan 6291234567890 # price and tax for barcodes
dotnet run -- replay 20          # recompute recent ERPNext invoices, report mismatches (read-only)
dotnet run -- parity <barcodes>  # draft invoice in ERPNext vs till totals (AllowWrites, dev only)
```

11. Tests

```
tests/TillPOS.Tests/
  Core/ Data/ Erp/ Sync/ Sync/Upload/ Printing/ Presentation/ App/
  Fakes/                      FakeErp (in-memory ERPNext), InMemoryCatalog, StubHandler (HTTP)
  Presentation/Fakes.cs       FakeDialogs, FakeNavigator, FakeClock, FakeOutput
  Presentation/PresentationFixture.cs  ready TillContext with sample cashiers, counters and items
  Data/TempDb.cs              a migrated SQLite file in %TEMP%
```

- Write the test first.
- Screen logic is tested through the view models with `PresentationFixture`; there are no UI-automation tests.
- Run a single class with `dotnet test -m:1 --filter FullyQualifiedName~PriceUpdateTests`.

12. Git workflow

- `master` is what is in production. Work on a branch (`feat/...` or `fix/...`).
- Stage files by name. Never `git add -A`, because `publish/` and databases must stay out.
- Before pushing, check for secrets: `git diff --cached` must contain no keys, secrets or PINs.
- Commit messages: `feat(area): …` or `fix(area): …`.

13. Troubleshooting

Symptom	Where to look
Till shows many **Waiting** and nothing uploads	Sync status → Failed tab. Often an earlier shift is still **Open** in ERPNext (POS Opening Shift list): close it there, then *Retry* in Upload problems.
Failed bill	Upload problems → View shows ERPNext's error. Fix the cause in ERPNext (price, tax, mode of payment, permission), then Retry.
Download feed error	Sync status → Download shows which feed failed. Usually a missing permission for the till's API user.
Crash or odd behaviour	`errors.log` next to `till.db` (each line starts with the version).
Price not updated	Wait 15 s or press F10. Check the Item Price belongs to the counter's price list.
PIN rejected	Section 8, *Checking a PIN problem*.